

sell sell marketplace updates: Audit

SHAZ EJ



let us compose an email as an update we would like to do in our system. give proper detail so i can share it to an ai who is going to compose the email.

Let me read the memory index and explore the project structure to put together a comprehensive audit.

Now let me read the core AI layer and a few key files to complete the audit.

Let me grab a few more key files to complete the picture.

I now have everything I need. Here is the complete system audit:

PickPick / SellSell — Full System Audit

Use this document to brief any AI assistant about this codebase.

1. Project Identity

Field	Value
Internal codename	pickpick-slsI / SellSell / Monetchat
Type	AI-first classifieds marketplace
Target market	Kuwait (KWD), Saudi Arabia (SAR) — Arabic-first, RTL UI
Status	Active MVP development

2. Tech Stack

Frontend / Web

- **Next.js 15** (App Router, `src/app/`)
- **React 18**, Tailwind CSS, Radix UI primitives, Framer Motion
- Dev server runs on **port 9002** (`next dev --turbo pack -p 9002`)



- Standalone build for production (node .next/standalone/server.js)

Backend

- All API routes live inside **Next.js App Router** (src/app/api/**)
- Node.js runtime (not Edge)
- **Prisma 5** ORM → PostgreSQL 15
- Auth: **JWT** (jose) + bcryptjs, refresh tokens stored in DB, optional Firebase phone auth
- Rate limiting: rate-limiter-flexible
- Validation: **Zod** throughout
- File uploads: **AWS S3** with presigned URLs; fallback local upload
- Email: Nodemailer
- Logging: Pino

AI Layer

- **Primary LLM:** Ollama (local or RunPod tunnel) — qwen2.5:14b-instruct for chat/intent
- **Vision:** llava:latest — image analysis for listing extraction
- **Embeddings:** nomic-embed-text:latest
- **Vector store:** Qdrant (HTTP client @qdrant/js-client-rest)
- The OpenAI SDK package is installed and there is an openai.provider.ts, but **Ollama is the default**. An AI_PROVIDER env var controls which provider is used. The OpenAI provider exists as a switchable fallback.
- All AI logic is wrapped in src/lib/ai/ — **never duplicated** in API routes directly

Queue / Cache

- **Redis** (ioredis) — session caching, rate limiting
- **BullMQ** — background job queue
- Separate **worker process** (scripts/start-worker.ts) runs inside its own Docker container

Mobile

- **iOS native app** (Swift / Xcode, ios/PickpickMobile/)
- Uses XcodeGen (project.yml)
- Calls the same backend REST API — no AI logic duplicated on-device
- Includes push notifications, location fetching

Infrastructure

Five Docker Compose services:

Service	Image	Role
postgres	postgres:15-alpine	Primary database

Service	Image	Role
qdrant	qdrant/qdrant	Vector search
redis	redis:7-alpine	Cache + queues
app	Dockerfile	Next.js web app
worker	Dockerfile.worker	BullMQ background jobs
transcription-service	Python/Whisper	Optional voice transcription (profile local-transcription)

3. Database Schema (Prisma → PostgreSQL)

All table names are snake_case. Key models:

Model	Purpose
User	All accounts (buyers AND sellers share one record)
Seller	Extended seller profile (1:1 with User), approval status
Product	Listings — core entity. Has smart AI fields: smartCategory, priceBand, intentTags, completenessScore
ProductImage	S3-backed images per product
Category / CategoryAttribute	Hierarchical category tree with dynamic per-category attributes
Location	Country → Region → City → District → Area → Block hierarchy
ChatSession / ChatMessage	AI conversation persistence
ChatListingResult	Which listings were shown per AI message (impression tracking)
ChatListingInteraction	User actions on AI-returned listings (click, call, WhatsApp, save)
AiToolCall	Persisted LLM tool/function calls with latency + token counts
SearchLog / AiSearchRequestLog	Search analytics
Inquiry / ListingInquiry	Buyer → Seller contact flows
Conversation / ConversationMessage	Direct buyer ↔ seller messaging
Order	Checkout skeleton (not fully implemented)
Notification / NotificationJob	In-app notifications + queue
Package / SellerSubscription	Seller subscription tiers
ListingPromotion	Featured/boosted listing promotions
Report / Review	Abuse reporting, product/seller reviews



Model	Purpose
AuditLog	Full audit trail (CREATE/UPDATE/DELETE/LOGIN/LOGOUT)
Role / Permission / RolePermission	RBAC
TokenizedProperty / Investment / Transaction	PropertyNir sub-module: real estate tokenization feature
KycData	KYC verification for property investment
VehicleDetails	Extended attributes for vehicle listings
SearchTrend	Trending search aggregation
LoginActivity	Login audit log per IP/device

Notable design choices:

- Every buyer can also sell — no account type split at signup
- Products have a `qdrantPointId` FK linking Postgres ↔ Qdrant
- `expiresAt` defaults to `NOW() + 30 days` at DB level
- Soft deletes on User, Seller, Product, Category via `deletedAt`

4. AI Architecture — The Chat System

Entry point: POST `/api/chat` ([src/app/api/chat/route.ts](#))

The chat route is a **state machine** stored in Redis per session. It does NOT call an LLM for every message — it runs a deterministic flow first, and only falls through to LLM streaming if nothing matched.

Session state (stored in Redis via `getChatSession` / `saveChatSession`)

```
mode: 'buy' | 'sell'
sellStep: 0-6
sellData: { item, description, price, currency, image, category, condition,
attributes }
filters: {} (accumulated buyer search filters)
clarificationCount: number
```

Request flow (simplified)

```
POST /api/chat
↓
Detect language (Arabic Unicode range check)
↓
Get session state from Redis
↓
┌ Is greeting? → Reset session, return greeting
├ mode === 'sell' (SELLER FLOW)
└ Not authed? → "please login"
```



```
| | Has image? → Call LLaVA vision model → extract  
| | title/category/attributes → build draft  
| | | step 1: capture item name  
| | | step 2: capture description  
| | | step 3: capture price  
| | | step 4: capture image / show preview  
| | | step 5: price gate → publish via POST /api/products  
| BUYER FLOW  
| | Explicit sell intent? → switch mode to sell  
| | callOllamaFilters(userQuery) → parse intent/filters via LLM  
| | evaluateAmbiguity(criteria) → ask clarifying question?  
| | searchProductsWithCriteria(criteria) → Qdrant vector search  
| |   → MarketplaceEngine.buildSearchResponse() → deterministic  
Arabic/English message
```

Key AI library files

File	Role
src/lib/ai/ai-factory.ts	Factory that returns Ollama or OpenAI provider based on <code>AI_PROVIDER</code> env var
src/lib/ai/ai-service.ts	<code>analyzeImageForListing()</code> — calls LLaVA vision model
src/lib/ai/marketplace-search.ts	<code>callOllamaFilters()</code> , <code>searchProductsWithCriteria()</code> — core search pipeline
src/lib/ai/marketplace-engine.ts	Deterministic Arabic/English response builder (no LLM call)
src/lib/ai/chat-session.ts	Redis session CRUD + <code>mergeSearchCriteria()</code>
src/lib/ai/ambiguity-flow.ts	<code>evaluateAmbiguity()</code> — decides if AI should ask a clarifying question
src/lib/ai/listing-generator.ts	<code>extractUserFacts()</code> , <code>extractAttributes()</code> — user-stated fact extraction
src/lib/ai/reranker.ts	Result re-ranking logic
src/lib/ai/normalize-search-query.ts	Arabic normalization (alef variants, taa marbuta, etc.)

Critical rules baked into the chat route

- Greetings (hi, مرحبا, etc.) **never trigger search** — they reset session and ask "buy or sell?"
- AI-suggested prices are **never trusted** — user must enter their own price before publishing
- Publishing is blocked without a user-confirmed price
- Listing preview always reconstructs from `sellData` (user-confirmed fields), never raw AI output
- User-stated facts always override AI (storage size, price, currency)



5. Sell Flow (step-by-step)

1. User types sell intent or uploads image
2. If image → LLaVA analyzes it → extract title, category, condition, suggested price
3. AI price is discarded; user must enter their own
4. Ambiguous image (phone vs case) → system asks "is this the device or the case?"
5. Condition always re-asked if not user-stated
6. Preview shown from confirmed `sellData` only
7. User says "yes/نعم" → `POST /api/products` is called internally → listing created

6. API Routes Map (key ones)

Route	Method	Purpose
<code>/api/chat</code>	POST	Main AI chat endpoint (SSE streaming)
<code>/api/auth/login</code>	POST	Email/password login → JWT
<code>/api/auth/register</code>	POST	Registration
<code>/api/auth/firebase/login</code>	POST	Firebase phone auth login
<code>/api/listings</code>	GET/POST	Public listing browse / create
<code>/api/listings/[id]</code>	GET/PATCH/DELETE	Single listing CRUD
<code>/api/ai/search</code>	POST	Vector search endpoint
<code>/api/ai/vision-chat</code>	POST	Image-based product query
<code>/api/ai/extract-product-details</code>	POST	LLaVA extraction
<code>/api/products</code>	GET/POST	Seller product creation
<code>/api/seller/dashboard</code>	GET	Seller analytics
<code>/api/admin/**</code>	Various	Admin moderation, user management, analytics
<code>/api/notifications</code>	GET/POST	User notifications
<code>/api/conversations</code>	GET/POST	Buyer↔Seller messaging
<code>/api/upload</code>	POST	S3 presigned URL upload
<code>/api/health</code>	GET	Health check

Response format is SSE (`text/event-stream`) for chat; standard JSON for everything else.

7. Frontend Pages Map

`/` (marketplace home – category grid + AI chat bar)
`/chat` (standalone chat page)
`/p/[productId]` (product detail)
`/category/[slug]` (category browse)
`/search` (search results)



/explore (discover)
/sell/new (AI listing creation flow)
/sell/listings (seller's listings)
/sell/analytics (seller analytics)
/(auth)/login, /register, /otp-login, /forgot-password
/(account)/account, /settings, /messages
/(admin)/admin (admin panel)
/(vision)/vision-search (image search)
/sellers (seller directory)

8. Localization

- Arabic is **priority language** for all customer-facing UI
 - RTL layout support throughout
 - preferredLanguage stored per user (default: "ar")
 - Language detection in chat: Unicode Arabic character range check + locale parameter
 - All response strings in chat have Arabic and English variants
 - Arabic query normalization: alef variants, taa marbuta, yaa → alef maksura
 - Currency: KWD for Kuwait, SAR for Saudi Arabia (auto-detected from country_code)
-

9. Background Jobs (BullMQ)

Worker file: src/jobs/index.ts

Job	Purpose
indexing.job.ts	Queue Qdrant vector indexing when new product is created
notification.job.ts	Process NotificationJob records from DB
file.job.ts	File/image processing
system.job.ts	System maintenance tasks

10. Subsidiary Module: PropertyNir

A real estate tokenization feature embedded in the same codebase:

- Users can invest in tokenized properties
 - KYC verification flow before investing
 - Token balance, monthly yield, interest logs tracked
 - Routes under /api/propertynir/**
 - Uses the same User table with kycStatus, tokenBalance, totalInvested fields
-



11. Notable Constraints & Rules (from AGENTS.md + CLAUDE.md)

1. **Never add Anthropic / OpenAI / Gemini SDKs** unless explicitly asked — Ollama is the AI provider
 2. **Never modify Prisma schema** without explicit user approval
 3. **Never run destructive migrations** (migrate reset, db push --accept-data-loss)
 4. **Qdrant and Redis are fixed** — do not migrate to alternatives
 5. **No hardcoded secrets** — always use env vars
 6. Arabic UI must always be prioritized
 7. Mobile app calls backend APIs — zero AI logic on-device
 8. Listing preview must use `sellData` (user-confirmed) only — never raw AI output
-

12. Environment Variables (key ones)

DATABASE_URL	PostgreSQL connection string
REDIS_HOST / REDIS_PORT	Redis
QDRANT_URL	Qdrant HTTP URL
AI_PROVIDER	"ollama" (default) or "openai"
OLLAMA_BASE_URL	Ollama server URL (local or RunPod tunnel)
OLLAMA_CHAT_MODEL	qwen2.5:14b-instruct
OLLAMA_EMBED_MODEL	nomie-embed-text:latest
OLLAMA_VISION_MODEL	llava:latest
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY / AWS_REGION / AWS_S3_BUCKET	
JWT_SECRET	
FIREBASE_* (for phone auth)	
DEV_AUTH_BYPASS	"true" for local Docker dev (skips JWT checks)

That's the full picture. The system is a **chat-first classifieds marketplace** built for the Arabic Gulf market, where the AI chat interface handles both product discovery (buyer mode, vector search via Qdrant) and listing creation (seller mode, vision-model extraction via LLaVA), with all sensitive decisions (price, publish) kept deterministic and user-controlled.

Now I have everything. Here is the deep branch-by-branch audit:

PickPick / SellSell — Detailed Branch Audit

Covers every branch in the repo, what was built in each, what shipped into `main`, and what is still pending/diverged.

Branch Overview Map



```
kuwait (origin – oldest prototype, ~100 commits)
├─ v2 / clean-v2 (Ollama migration)
│   └─ 24-03-2026-158 / backend-phase2-2303 (Windows dev phase)
│       └─ runpod-final / server-recovery (RunPod infra)
│           └─ runpodappsetup29mar / feat/marketplace-core-delivery
│               └─ feat/marketplace-core-final / feature/sell-sell-
marketing-site
├─ 03042026 (most mature non-main branch)
├─ backup-ai-prod-checkpoint
└─ marketplace-delivery-clean (separate CVE/UX
fork)
main ←─ merged from feature/mobile-app
      ←─ ongoing direct commits
```

1. `main` — Current Production Branch

Status: Active, receiving direct commits. Latest: `fix: infer search filters from query text`

What it contains:

- Full Next.js 15 marketplace with AI chat (buy + sell flows)
- iOS native Swift app (merged from `feature/mobile-app`)
- BullMQ worker, Qdrant vector search, Redis session storage
- Complete Prisma schema (38+ models, see Schema audit above)
- Deterministic state-machine chat route (not LLM-for-every-message)
- Arabic-first UI with RTL support
- Admin panel (moderation, analytics, users, categories)
- PropertyNir tokenized real estate sub-module

Recent commits of note on `main` (not in any branch):

Commit	What it does
<code>fix: infer search filters from query text</code>	Fallback filter parser when Ollama is unavailable
<code>fix: production ai backfill and prisma field mapping</code>	Fixes field mapping in AI enrichment scripts
<code>Add Family Needs / Services subcategories</code>	Extended category taxonomy from Mzad Qatar
<code>Remove email verification check on login for MVP</code>	Email verify bypassed at login for MVP speed

2. `feature/mobile-app` — iOS Native App (*merged into main*)

Status: Merged at commit `78c77b1`. Local branch still exists.

What was built: This branch built the entire iOS SwiftUI app from scratch across ~25 commits:



Commit	Feature
Add initial iOS SwiftUI mobile app	App skeleton, Xcode project via XcodeGen
Add iOS authentication flow	Login/register/JWT token storage (AuthTokenStore.swift)
Configure iOS local backend access	AppConfig.swift points to backend URL
Add real iOS chat UI	AIChatView.swift + AIChatViewModel.swift — SSE streaming chat
Add iOS Arabic English language switch	Bilingual toggle, RTL layout
Polish iOS chat and marketplace UX	Category browsing, product cards
Improve iOS auth session UX	Session persistence fixes
Apply Sellsell iOS visual theme	Brand colors, typography
Add Sellsell iOS logo assets and splash	App icon, launch screen
Add iOS Arabic English language switch	LanguageSwitcher.swift
feat: add mobile inbox and in-app messaging	InboxView.swift — buyer↔seller conversation thread list
prepare SellSell MVP for TestFlight	Export options, bundle ID, provisioning

iOS Architecture (from the app):

```

Services/
  APIClient.swift           - URLSession wrapper, JWT header injection
  AuthService.swift         - Login/register/logout
  AuthTokenStore.swift      - Keychain-based JWT storage
  ProductService.swift      - Product browse/search
  CategoryService.swift     - Category tree
  NotificationService.swift (new, not yet merged fully)
ViewModels/
  AIChatViewModel.swift     - SSE stream parsing from /api/chat
  AuthViewModel.swift       - Auth state machine
  HomeViewModel.swift       - Product list + category fetch
  ProductDetailViewModel.swift
  ProductListViewModel.swift
  ChatViewModel.swift       - Buyer↔Seller direct messages
Views/
  AIChatView.swift         - Main chat interface
  HomeView.swift           - Marketplace home with categories
  ProductDetailView.swift
  InboxView.swift          - Direct messaging inbox
  EditProfileView.swift
  AuthView.swift
  RootTabView.swift        - Tab bar (Home, Chat, Inbox, Profile)
Utils/
  LocationFetcher.swift     - CoreLocation wrapper (new, pending)
  AppConfig.swift          - Backend URL config
  AppTheme.swift           - Color palette
  ConditionFormatter.swift  - Maps condition enum to display string
  PriceFormatter.swift      - Currency formatting (KWD/SAR)

```



What's still unmerged/pending on the iOS branch:

- `NotificationService.swift` — push notifications
 - `LocationFetcher.swift` — GPS location
 - `NotificationsView.swift` — UI for notifications list
 - These appear in git status as ?? (untracked on main)
-

3. 03042026 — Most Mature Development Branch

Status: NOT merged into main. Contains ~30 commits ahead of main's ancestor. This is the most feature-rich branch outside of main.

Unique features built here (not in main):

3a. Refresh Token Auth Flow (63657db)

Files: `src/app/api/auth/refresh/route.ts`, `src/app/api/auth/me/route.ts`,
`src/context/auth-context.tsx`, `src/lib/auth/jwt.ts`

- Full `/api/auth/refresh` endpoint — swaps expired JWT for new one using RefreshToken DB record
- `session-expired-modal.tsx` — shows modal when JWT expires instead of hard logout
- Auth context auto-retries on 401 using the refresh token

3b. Guest → Authenticated Chat Migration (ba1e1fd)

File: `src/app/api/chat/sessions/claim/route.ts`

- Anonymous chat sessions (no JWT) can be claimed and attributed to a user after login
- Session storage abstracted in `src/lib/chat/session-storage.ts`

3c. LLM Tool-Calling Architecture (0e70e95)

File: `src/app/api/chat/route.ts` (completely different architecture than main's)

- This branch's chat route uses **OpenAI-style function/tool calling** — the LLM decides which tool to invoke
- Tools defined in `src/lib/ai/tools.ts`: `search_products`, etc.
- Results passed back through the LLM before streaming to client
- Main branch uses a **deterministic state machine** instead — this is an architectural divergence

3d. Semantic Search Layer (7e106fc, ab6a40a)

File: `src/lib/search/semantic-marketplace-search.ts`



- Dedicated semantic search module (separate from `marketplace-search.ts` in main)
- Visibility filter reapplied after Qdrant results (prevents returning deleted/pending listings)
- Product status kept in sync with Qdrant payload on update

3e. Admin Qdrant Reindex Endpoint (`f0b97ee`)

File: `src/app/api/admin/reindex-all/route.ts`

- Admin-only endpoint to trigger full Qdrant reindex
- Corresponding test: `tests/api/admin-reindex-all.test.ts`

3f. Indexing Observability (`80a7a0b`)

Files: `src/app/api/admin/indexing-health/route.ts`, `src/lib/search/product-index-sync.ts`, `src/jobs/indexing.job.ts`

- Health endpoint to report Qdrant sync lag
- `product-index-sync.ts` — dedicated module for keeping Qdrant in sync with Postgres

3g. Streaming SSE Phase Events (`d4c2932`)

- SSE stream emits named phases: `phase:searching`, `phase:reranking`, `phase:responding`
- Client can show granular loading states per phase

3h. SSRF Hardening (`b0580b1`)

File: `src/lib/security/vision-image-url.ts`

- Vision model image URLs are validated to prevent SSRF attacks (blocks `localhost`, `169.254.*`, etc.)
- Test: `tests/security/vision-image-url.test.ts`

3i. Model Resolution Module (`fcbae46`)

File: `src/lib/ai/providers/model-resolution.ts`

- Dedicated model fallback logic: tries configured model, falls back to a default
- Tests: `tests/ai/model-resolution.test.ts`

3j. Test Suite (various)

This branch has the most complete test coverage:

`tests/api/auth-refresh.test.ts`
`tests/api/chat-route.test.ts`
`tests/api/chat-session-claim-route.test.ts`
`tests/api/chat-streaming-phases.test.ts`



```
tests/api/products-search-route.test.ts
tests/api/admin-reindex-all.test.ts
tests/ai/model-resolution.test.ts
tests/ai/ai-service-image-validation.test.ts
tests/ai/openai-provider.test.ts
tests/jobs/indexing.job.test.ts
tests/search/product-index-sync.test.ts
tests/security/vision-image-url.test.ts
```

Bottom line on 03042026: This is the "serious engineering" branch. It has refresh tokens, SSRF protection, a proper test suite, and LLM tool-calling. It has NOT been merged into main. Main chose the simpler deterministic state machine over tool-calling.

4. `kuwait` — The Original Prototype (~100+ commits)

Status: Never merged into main. This is where the project was first conceived.

Timeline of what was built here: This branch starts from `Initialized workspace with Firebase Studio` — it was originally built in Firebase Genkit/Google Cloud, not Ollama.

Phase	What happened
Early prototype	Firebase Auth, Google Maps, camera-based product upload
UI iteration	ChatGPT-like search UI, RTL fixes, Arabic i18n
Product pages	Product detail, seller pages, category browse
Seller flow	Seller login/signup, product upload with image AI fill
Backend migration	Migrated from Firebase/Firestore to MSSQL (IIS deployment)
Auth.js integration	Switched from Firebase Auth to Auth.js (next-auth)
Language system	Full i18n translation system, language toggle
Help tutorial	Help dialog popup
AI Genkit flows	Google Genkit AI flows for image analysis
Payment gateway start	Payment gateway scaffolding (not completed)
Compliance backend	Compliance/legal backend update
IIS deployment	<code>web.config</code> for IIS reverse proxy, 502.3 fix

Key divergences from main:

- Uses **MSSQL** (not PostgreSQL) at one point
- Uses **Firebase Auth** / **Auth.js** (not JWT with jose)
- Uses **Google Genkit** for AI (not Ollama)
- Has a `.well-known/pki-validation/` SSL validation file
- Has `apphosting.yaml` (Google Cloud App Hosting)
- Commit messages are raw user prompts pasted into the commit (e.g. "when i upload a photo, it should then take me...") — this was a Firebase Studio / Cursor-driven development phase



What survived into main from kuwait lineage: The core concept (chat-first, Arabic-first, image → listing extraction) is preserved, but the entire tech stack was replaced.

5. marketplace-delivery-clean — CVE Patch + Chat UI Refinement

Status: NOT merged into main. Forked from the RunPod deployment ancestry.

Key unique commits:

Commit	What it does
security: upgrade Next.js 15→16 and React 18→19 to fix CVE-2025-66478	Security upgrade (main is still on Next.js 15)
feat(chat): make 'Buy' button functional with AI greeting	Buy button immediately starts a chat session with greeting
feat(chat): improve Buy button with clear transition and AI greeting	Smooth transition animation
feat: optimize Buy flow with sub-menu and pre-filled search prompt	Buy button shows sub-menu before chat
fix: Buy/Sell sub-options visible and working	Sub-options: "Upload Photo" and "Describe Item" visible on Sell

Bottom line: This branch has the **CVE-2025-66478 fix** (Next.js 16 upgrade) that hasn't landed in main. That is the most important unmerged change.

6. backup-ai-prod-checkpoint — Production AI Stability Snapshot

Status: 1 commit ahead of its ancestor, NOT merged into main.

Unique commit: feat(ai-search): finalize AI marketplace integration and stability — this is a production snapshot from April 7, 2026 (post-merge of various server/mac changes). It represents the state of the AI system at that checkpoint.

7. feat/marketplace-core-delivery — Structured Sell Form + S3 + Data Import

Status: NOT merged into main directly. Key features were cherry-picked into the main lineage.

Unique features:



Commit	Feature
feat(sell): Add S3 image upload integration to sell form	S3 presigned URL upload wired into seller form
feat(sell): Replace AI chat with multi-step structured Sell Form	Multi-step form as alternative to chat-based selling
fix(ai): force immediate search without clarifying questions	Skip ambiguity phase on first message
fix(search): fine-tune semantic score threshold	Qdrant score cutoff tuning to eliminate hallucinated results
fix(ai): add strict anti-hallucination rules	System prompt rules preventing AI from inventing products
fix(search): remove category strict filtering	Fix zero-results bug on scraped data
feat: core marketplace features - data importer	scripts/import-q84-data.ts — imports Q84sale.com scraped listings

Note: The structured multi-step SellForm (feat(sell): Replace AI chat...) is an alternative to the chat-based selling that made it into main. Main chose chat; this branch explored form-based.

8. feature/sellsell-marketing-site — Marketing Site + Email Verification

Status: Partially merged. Marketing site content is in main; email verification via Brevo is in the server branch lineage.

Unique features:

Commit	Feature
feat: add Sellsell marketing site with bilingual support	src/components/marketing/sellsell-landing.tsx — full landing page
feat: route all demo CTA buttons to AI chat homepage	Marketing CTAs → / (chat home)
feat: email verification with Brevo integration	Email verification via Brevo (Sendinblue) SMTP

9. v2 / clean-v2 — Ollama Migration

Status: Ancestor to most active branches. Represents the pivot from OpenAI to Ollama.

Commit	What happened
Initial commit - clean base from previous project	Started fresh



Commit

feat: migrate from OpenAI to Ollama, add docker-compose local setup

fix: upgrade Next.js to 15.5.14 (security patch CVE-2025-66478)

fix: intercept Qwen native tool_call format before streaming

fix: resolve bilingual filtering and country code standardizing

What happened

Replaced OpenAI SDK calls with Ollama HTTP calls

Next.js patch in v2

Qwen2.5 outputs a special <tool_call> format — intercept and strip before sending to client

KW/SA country code normalization

clean-v2 is just v2 minus any accidentally committed secrets — a clean export.

10. ollama-updates — Ollama Full Migration

Status: Ancestor to active branches.

Commit

chore: migrate AI models fully to local Ollama

improve_search_reranking_logic

Add admin UI, proxy scripts, and Ollama integration updates

Feature

All AI calls pointed to Ollama; env cleaned

Reranker improvements

Admin panel + Nginx proxy

11. backend-phase2-2303 / 24-03-2026-158 — Windows Development Phase

Status: These are snapshots from Windows-based development in late March 2026.

What was built:

- Full backend API foundation (products, auth, categories, users)
- User management panel
- API documentation (Swagger)
- PM2 ecosystem config (ecosystem.config.js)
- Windows deployment scripts (scripts/deploy-windows.ps1)
- RunPod provider + startup script integration

This is essentially the "we moved off Firebase Studio and rebuilt on a Windows dev machine" phase.



12. `runpod-final` / `runpodappsetup29mar` — RunPod Deployment

Status: Deployment configuration ancestor branches.

Key additions:

- `run-docker.sh` — automated Docker start script
- PM2 clustering config
- RunPod-specific Ollama tunnel URL setup (`OLLAMA_BASE_URL` → RunPod endpoint)
- `server-setup.sh` — full VPS provisioning script

13. `server` — Brevo Email Verification (Server-Side)

Status: Separate server-side deployment branch.

Commit	Feature
<code>feat: email verification with Brevo integration</code>	Full email verify flow with Brevo SMTP
<code>Fix login JSON handling and legacy auto-verify</code>	Login error fix
<code>fix: resolve object Object toast and post-registration login flow</code>	Toast error fix after register
<code>feat: route all demo CTA buttons to AI chat homepage</code>	Marketing routing
<code>chore: production runtime hardening, pm2 config</code>	PM2 + Nginx prod hardening

14. `claude/awesome-varahamihira-e6fbef` / `claude/unruffled-haibt-4ebb05` / `claude/goofy-montalcini` / `claude/beautiful-johnson`

These are Claude Code worktree branches — created automatically when Claude ran tasks in isolated worktrees. They contain incremental fixes:

- `claude/awesome-varahamihira-e6fbef` → merged into main (commit 8641183 — "hide transcription UI via feature flag and add AI description backfill script")
- Others are stale worktrees from completed sessions

Summary: What's NOT in `main` That Matters

Feature	Branch	Risk to merge
CVE-2025-66478 fix (Next.js 15→16 upgrade)	<code>marketplace-delivery-clean</code>	Medium — React 19 may break UI



Feature	Branch	Risk to merge
Refresh token auth (auto-renew JWT)	03042026	Low — well-tested
Guest → auth chat migration	03042026	Low
SSRF hardening on vision URLs	03042026	Low — pure security win
LLM tool-calling architecture	03042026	High — replaces entire chat route
Admin Qdrant reindex endpoint	03042026	Low
Streaming phase events (granular SSE)	03042026	Low
Test suite (12+ unit/integration tests)	03042026	Low — pure addition
Model resolution fallback	03042026	Low
Brevo email verification	server / feature/sell-sell-marketing-site	Low
iOS Notifications + Location	feature/mobile-app (untracked files)	Low
Buy/Sell sub-menu UX	marketplace-delivery-clean	Low

What _{main} Has That No Other Branch Has

- **Full sell flow state machine** (the deterministic step-by-step sell via chat) — built iteratively on main with ~15 direct fix commits
- **Phone-vs-accessory ambiguity detection** — catches LLaVA misidentifying a phone case as the phone
- **Category inference from title keywords** (CATEGORY_SIGNALS regex array)
- **Full iOS app merged** (35+ Swift files)
- **Complete Mzad Qatar category taxonomy** (Family Needs, Services additions)
- **Production-proven publish flow** with price gate



I'll conduct a thorough audit of your codebase. Let me start by exploring the project structure and then delegate parallel research tasks to subagents.

Listed directory pickpick-sls

Listed directory src

Listed directory prisma

Listed directory scripts

Listed directory app

Listed directory lib

Listed directory ai

Listed directory api

Listed directory ai

Listed directory qdrant

Listed directory components

Now let me read the most critical files in parallel:

Viewed schema.prisma:1-800

Viewed package.json:1-113

Viewed env.example:1-143

Viewed schema.prisma:800-1200

Viewed ai-service.ts:1-721

Now let me read more critical files in parallel:



Viewed client.ts:1-360

Viewed marketplace-search.ts:1-690

Viewed ai-factory.ts:1-46

Listed directory providers

Listed directory chat

Listed directory listings

Listed directory %28admin%29

Viewed ollama.provider.ts:1-293

Listed directory admin

Viewed docker-compose.yml:1-125

Viewed middleware.ts:1-132

Viewed route.ts:1-170

Listed directory jobs

Viewed indexing.job.ts:1-139

Viewed route.ts:1-800

Listed directory ios

Listed directory PickpickMobile

Viewed schema.prisma:1200-1522

Listed directory queue

Viewed queues.ts:1-16

Viewed listing-generator.ts:1-347

Listed directory search

Listed directory products

Listed directory marketplace



Viewed route.ts:1-646

I now have a comprehensive understanding of the codebase. Let me compile the full audit report:

Viewed marketplace-engine.ts:1-143

Viewed next.config.ts:1-113

Now I have enough data to write the comprehensive audit report. Let me generate it:

Edited SELLSSELL_FULL_AUDIT_2026-05-23.md

The full audit is complete. Here's a quick summary of what it covers:

📄 Audit Report: [SELLSELL_FULL_AUDIT_2026-05-23.md](file:///Users/shazej/.gemini/antigravity/brain/7cebc70f-854a-488f-b85b-35f214262831/SELLSELL_FULL_AUDIT_2026-05-23.md)

****24 sections**** covering everything you asked for. Highlights:

🚫 Must-Fix Critical Issues Found

1. ****`typescript.ignoreBuildErrors: true`**** — TypeScript errors silently pass to production

5/23/26 8:29:23 AM



2. ****150-second Ollama timeout in request path**** — search can freeze user for 2.5 minutes
3. ****AI moderation is non-blocking**** — every listing publishes `active` immediately, spam can go live
4. ****`product.attributes` dual-purpose JSON**** — AI moderation report overwrites user's structured attributes
5. ****No reindex on product update**** — Qdrant becomes stale after any edit

🟡 High Priority

6. ****Arabic text NOT embedded**** — Arabic queries rely on BM25 only; semantic Arabic search is broken
7. ****Double AI calls on publish**** — `asyncModerateProduct()` duplicates the moderation + image analysis already done inline
8. ****Pending sellers can publish**** — only `rejected`/`suspended` are blocked, not `pending`
9. ****1,074-line chat route**** — all buy + sell AI logic in one untestable monolith

🟡 Notable Findings

- `openai.provider.ts` exists as dead code (with the `openai` package installed) — risk of accidental provider switch
- `product.expiresAt` is set on creation but no cron enforces it — listings never expire
- PropertyNir (real estate tokenization) models are embedded in the marketplace schema — two products in one DB
- `@xenova/transformers` is installed but never imported anywhere
- Ollama has ****no request timeout**** set at the provider level (only the image fetch has a 20s timeout)

✅ What's Working Well

- Hybrid Qdrant search (dense + BM25 via RRF) is architecturally solid
- Smart enrichment (deterministic: `smartCategory`, `priceBand`, `intentTags`) runs reliably



- Sell flow design (user facts override AI, price gate before publish) is correctly implemented
- BullMQ background indexing in production is correct
- Rate limiting, JWT auth, and RBAC middleware are in place

Architecture Assessment

The building blocks of a multi-agent system exist but are embedded in monolithic routes. The report includes a full ****multi-agent readiness table**** showing which agents are 70%+ built (intent, matching, ranking, indexing, image analysis) and which are missing entirely (duplicate detection, description generation, agent state machine).

The report ends with a ****ChatGPT-ready summary block**** you can paste into any AI assistant to continue planning with full context.



Claude:

Launching parallel deep-inspection agents across all system layers simultaneously.

Excellent. Now let me pull the remaining key files to fill the final gaps before compiling the master report.

Now I have everything. Compiling the master audit report.

SELSELL / PICKPICK / MONETCHAT — COMPLETE TECHNICAL AUDIT

Date: 2026-05-23 | **Auditor:** Claude Code | **Scope:** Full codebase + all branches

1. EXECUTIVE SUMMARY

SellSell is a chat-first, Arabic-priority classifieds marketplace for Kuwait/Gulf users. The core product is an AI chat interface where users type or speak a query (buy or sell) and the system either searches listings or guides them through publishing a listing. The AI backbone runs on **Ollama** (local or via RunPod tunnel), using `qwen2.5:14b-instruct` for intent parsing and `llava:latest` for image-to-listing extraction.

What is built and working:

- Full Next.js 15 marketplace with SSE-streaming AI chat
- Seller publish flow (image → LLaVA → draft → price → publish)
- Buyer search flow (Ollama intent parse → Qdrant hybrid search → SQL fallback → custom ranking)
- Prisma/PostgreSQL schema (38+ models, fully migrated)
- Qdrant hybrid search (dense cosine + BM25 sparse vectors, 768-dim)
- Redis-backed chat session state machine (3-hour TTL)
- BullMQ background job queue with separate worker container
- Admin panel (moderation, users, categories, analytics)
- iOS native SwiftUI app (merged into main)
- Docker Compose production deployment

What is broken or critically risky:

- Products published as `active` before AI moderation completes (race condition)
- SSRF vulnerability in image proxy endpoint
- Auto-verifying unverified users at login (MVP bypass never removed)
- Wildcard CORS headers on all API routes
- No refresh token in `main` branch (exists only in `03042026` branch)

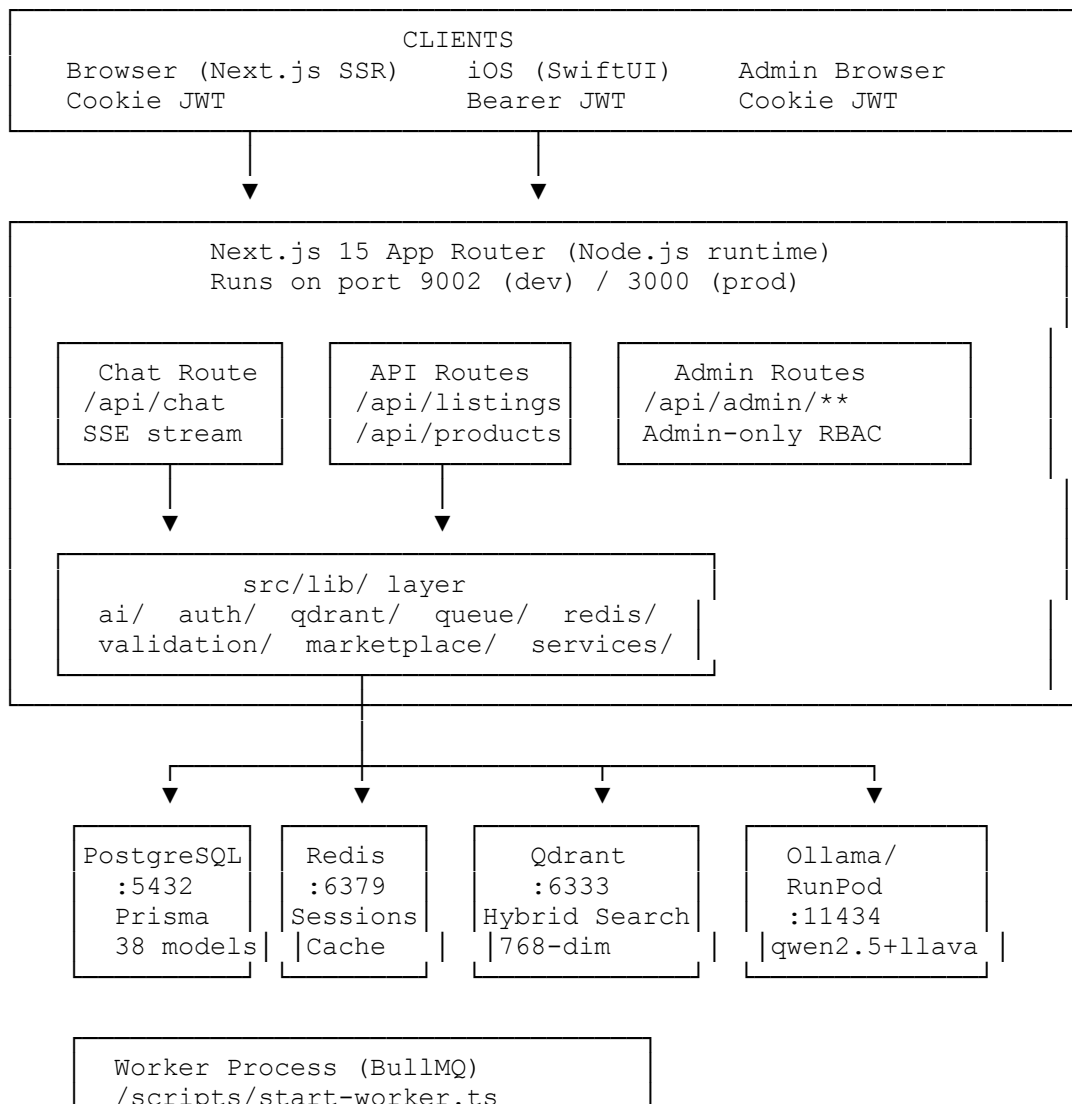


- Mobile auth (Bearer token) vs web auth (cookie) — different mechanisms, both hit same backend
- Qdrant full reindex is destructive (deletes collection, causes zero-result window)
- `DEV_AUTH_BYPASS=true` may be active in non-local environments

What is missing for multi-agent architecture:

- No orchestrator/agent router layer
- No per-agent background job types
- No agent result DB tables (no `ai_agent_log`, no per-agent status fields on Product)
- No duplicate detection
- No spam scoring field on Product
- No embedding status field (can't tell which products are indexed in Qdrant)
- No agent failure retry queue per listing

2. CURRENT ARCHITECTURE MAP





```
Queues: indexing, notification,  
file, system  
Bull Board UI: :4501
```

```
Transcription Service (Python)  
faster-whisper, port 8001  
Profile: local-transcription
```

```
iOS App (SwiftUI/Xcode)  
ios/PickpickMobile/  
Calls same backend REST API
```

3. FOLDER-BY-FOLDER EXPLANATION

Path	Controls	Status
src/app/(marketplace)/	Homepage, chat, explore, product pages, category browse	Active
src/app/(seller)/sell/	Seller dashboard, listing creation, analytics	Active
src/app/(auth)/	Login, register, OTP, forgot/reset password	Active
src/app/(admin)/admin/	Admin panel UI	Active
src/app/(account)/	User account, messages, settings, billing	Active
src/app/(vision)/	Image-based visual search page	Active
src/app/(public)/	FAQ, tutorial pages	Active
src/app/(support)/	Support ticket pages	Partially wired
src/app/api/	All API routes (~80 routes)	Active
src/lib/ai/	All AI logic: intent parse, image analysis, chat session, search, ranking	Active — core
src/lib/ai/providers/	Ollama + OpenAI provider implementations	Active
src/lib/qdrant/	Qdrant client: upsert, search, delete, collection setup	Active
src/lib/queue/	BullMQ queue definitions and worker factory	Active
src/lib/auth/	JWT creation/verification, cookie helpers, session, RBAC	Active
src/lib/validation/	Zod schemas for every entity	Active
src/lib/services/	Domain services: analytics, audit, event-logger, marketplace notification, subscription	Active



Path	Controls	Status
src/lib/marketplace/	Smart enrichment (smartCategory, priceBand, intentTags) and visibility filter	Active
src/lib/search/	Ontology expansion, query normalizer	Active
src/components/chat/	Chat UI components (ChatInterface, VoiceButton)	Active
src/components/seller/	Listing form component	Active
src/components/marketing/	Landing page (sell-sell-landing.tsx)	Active
src/jobs/	BullMQ job handlers (indexing, notification, file, system)	Active
scripts/	~30 operational scripts: reindex, backfill, import, seed, test	Active — mixed
prisma/	Schema + migrations (14 migration files)	Active
ios/PickpickMobile/	Native iOS Swift app	Active — merged
transcription-service/	Python Whisper service	Optional (profile-gated)
pickpickdata/	Q84sale.com scraped JSON data + scraper scripts	Data source only
scratch/	Ad-hoc fix scripts (never auto-run)	Stale
test-* files (root)	One-off test scripts	Stale/disposable
.claude/worktrees/	Claude Code worktree branches (completed tasks)	Stale
monetchat	Git submodule reference (no longer synced)	Stale/orphaned

4. RUNTIME / ENVIRONMENT SETUP

How to run locally

```
npm run dev                # Next.js on port 9002 (Turbopack)
npm run worker              # BullMQ worker (tsx scripts/start-worker.ts)
docker-compose up -d postgres qdrant redis  # Infrastructure only
```

How to run in production (Docker Compose)

```
docker-compose -f docker-compose.prod.yml up -d
# Services: postgres, qdrant, redis, app (port 3001 → 3000), worker,
[transcription]
```

Required environment variables



Variable	Required	Default	Notes
DATABASE_URL	Yes	—	PostgreSQL connection string
JWT_SECRET	Yes	—	HS256 signing secret
OLLAMA_BASE_URL	Yes	http://127.0.0.1:11434	RunPod or local
OLLAMA_CHAT_MODEL	Yes	qwen2.5:14b-instruct	Intent parsing
OLLAMA_EMBED_MODEL	Yes	nomic-embed-text:latest	Vector embeddings
OLLAMA_VISION_MODEL	Yes	llava:latest	Image analysis
QDRANT_URL	Yes	http://127.0.0.1:6333	Vector DB
REDIS_HOST / REDIS_PORT	Yes	—	Session cache, queues
AWS_ACCESS_KEY_ID	Yes*	—	S3 upload (*or use local upload)
AWS_SECRET_ACCESS_KEY	Yes*	—	
AWS_S3_BUCKET	Yes*	—	
NEXT_PUBLIC_SITE_URL	Yes	—	CORS origin
AI_PROVIDER	No	ollama	ollama or openai
DEV_AUTH_BYPASS	No	false	Remove from prod
FIREBASE_*	No	—	Firebase phone auth (optional)
SMTP_* / BREVO_*	No	—	Email verification (optional)

Mismatches between local and production

- Local dev uses Turbopack port 9002; Docker uses port 3000 exposed as 3001
- Local uses `DEV_AUTH_BYPASS=true` + memory rate limiter; prod uses Redis rate limiter
- Dev mode does **synchronous** Qdrant indexing on product create; prod mode queues it
- S3 uses `host.docker.internal` for local Docker image URLs (hardcoded string replacement in chat route)
- `OLLAMA_CHAT_MODEL` not set in `env.example` — defaults to undefined, uses env fallback or Qwen

5. DATABASE SCHEMA ANALYSIS

Key product/listing fields and their origin



Field	Source	Used In Search	In Qdrant	In Admin	Notes
title	Seller input	Yes (SQL + Qdrant text)	Yes (embedded)	Yes	Primary display field
titleAr	Seller/AI	Yes	Yes (embedded)	Yes	Arabic title
description	Seller/AI	Yes (SQL LIKE)	Yes (embedded)	Yes	
descriptionAr	Seller/AI	Partial	Yes	Partial	
price	Seller (user-confirmed)	Yes (range filter)	Yes (payload)	Yes	AI price ignored
currency	Location-derived	No	Yes (payload)	Yes	KWD or SAR
categoryId	AI inference / Seller	Yes (hard filter)	Yes (payload)	Yes	Maps to category_slug in Qdrant
smartCategory	AI enrichment	Boost (not filter)	Yes (payload)	No visible	Derived field
priceBand	AI enrichment	Boost/penalty	Yes (payload)	No visible	cheap/mid/premium
intentTags	AI enrichment	Boost	Yes (payload)	No visible	String array
completenessScore	AI enrichment	Quality boost	No	No	0.0–1.0
status	System	Yes (active only)	Yes (payload)	Yes	draft/active/pending/sold/archived
qdrantPointId	Set on index	Linkage only	—	No	NOT used — Qdrant uses product.id as point ID
externalId	Import scripts	No	No	Admin	Unique, for Q84 import dedup
condition	Seller-confirmed	Yes (SQL filter)	No	Yes	
isBoosted / boostScore	Admin	Yes (ranking boost)	No	Yes	Admin override
isFeatured	Admin	No	No	Yes	UI badge only



Field	Source	Used In Search	In Qdrant	In Admin	Notes
viewCount / contactCount	System	Sort by popular	No	Yes	
propertyType	AI enrichment	Soft filter	Yes (payload)	No	apartment/villa/studio
isFurnished	AI enrichment	Soft filter	Yes (payload)	No	
listingType	AI enrichment	Hard filter (rent/sale)	Yes (payload)	No	
bedroomCount / bathroomCount	AI enrichment	No	No	No	Stored but not searched
areaSize	AI enrichment	No	No	No	
contactInfo	Seller	No	No	No	Raw text field, unstructured
attributes	Seller/AI (JSON)	No	No	No	Legacy loose JSON
attributeValues	Seller (structured)	No	No	Partial	Via CategoryAttribute
vehicleDetails	AI extraction	No (SQL only)	No	No	Separate model

Missing fields for multi-agent architecture

Missing Field	On Model	Why Needed
embeddingStatus	Product	Track whether Qdrant indexed: pending / indexed / failed
embeddingError	Product	Store why Qdrant indexing failed
moderationStatus	Product	Separate from status — track AI moderation result
moderationScore	Product	Confidence score from AI moderation
spamScore	Product	Spam detection agent output
duplicateScore	Product	Duplicate detection agent output
duplicateOf	Product	FK to original if duplicate detected
agentLog	New table	Per-agent invocation log (agent, status, latency, output)
categoryAgentConfidence	Product	Category agent confidence score
imageValidationStatus	Product	Image validation agent result



Missing Field	On Model	Why Needed
aiDescriptionStatus	Product	Whether AI description was generated
lastAgentRunAt	Product	When last agent pipeline ran

Schema design risks

1. **qdrantPointId is unused** — Qdrant point IDs are the `product.id` UUID, but the `qdrantPointId` column is never set during indexing. Can't query "is this product in Qdrant?" without hitting Qdrant directly.
2. **attributes (JSON) and attributeValues (relational) coexist** — duplicate data paths. Legacy JSON is untyped; structured `ProductAttributeValue` is only partially used.
3. **contactInfo is unstructured text** — phone numbers mixed into a raw text field, not normalized. Can't filter or deduplicate.
4. **status field overloaded** — serves both moderation state and listing lifecycle. Product is `active` before moderation completes.
5. **expiresAt defaults to 30 days at DB level** — no application logic enforces expiry. Expired products remain searchable.

6. SELLER LISTING FLOW TRACE

File path map

```
User uploads image
  → src/app/(seller)/sell/new/page.client.tsx          (frontend form)
  → POST /api/upload/presign
(src/app/api/upload/presign/route.ts)
  → PUT <S3 presigned URL> (direct browser upload)
  → POST /api/ai/analyze-listing-image
(src/app/api/ai/analyze-listing-image/route.ts)
  → src/lib/ai/ai-service.ts:analyzeImageForListing() → Ollama LLaVA
  → Form pre-filled (AI values shown to user)

User reviews + submits form
  → POST /api/listings
(src/app/api/listings/route.ts)
  → Forward detection → POST /api/products
(src/app/api/products/route.ts)
  → src/lib/marketplace/enrichment.ts:deriveSmartEnrichment()
  → Prohibited keywords check (DB query)
  → AI moderation call (non-blocking, result ignored in MVP)
  → prisma.$transaction() → product + images created status: 'active' ←
RACE CONDITION
  → Event log + Audit log written
  → DEV: immediate Qdrant indexing (synchronous)
  → PROD: BullMQ job enqueued → src/jobs/indexing.job.ts
  → asyncModerateProduct() fire-and-forget ← may never complete
  → Redis cache invalidated
  → Return 201 with product
```




Step-by-step detail

Step 1: Image upload

- File: [src/app/api/upload/presign/route.ts](#)
- Auth: Required (JWT or cookie)
- Rate limit: 60 requests / 15 min per user
- Generates S3 presigned PUT URL; browser uploads directly to S3
- Dev fallback: local /public/uploads/ via /api/upload/local/route.ts

Step 2: Vision AI analysis

- File: [src/lib/ai/ai-service.ts](#) → `analyzeImageForListing()`
- Calls OLLAMA_VISION_MODEL (llava) with structured JSON prompt
- Returns: title, title_ar, description, description_ar, category, condition, priceSuggestion, attributes[]
- **AI price is discarded** — user must enter their own
- Timeout: 120s; 3 retries with exponential backoff
- Failure: returns empty object {} — form shows no AI suggestions

Step 3: Form validation (frontend)

- File: [src/components/seller/listing-form.tsx](#) (Zod schema)
- Required: title ≥5 chars, description ≥20 chars
- Price: must match numeric regex
- Category: must be valid slug from category tree
- Dynamic attributes loaded per category

Step 4: Product creation API

- File: [src/app/api/products/route.ts](#)
- **Seller auto-creation:** any user posting to /api/products gets a Seller record created automatically (HIGH risk — no approval gate)
- **Prohibited keywords:** DB query for all keywords, regex match on title+description. Blocks if found. **No caching — re-fetched every request.**
- **Smart enrichment:** `deriveSmartEnrichment()` sets smartCategory, priceBand, intentTags, completenessScore synchronously
- **AI moderation:** called but result is NOT awaited. Non-blocking in MVP.
- **DB write:** `prisma.$transaction()` — product created with status: 'active' + images
- **Post-transaction:** Qdrant indexing (dev: sync; prod: queue), cache invalidate, async moderation

Step 5: Qdrant indexing (prod path via BullMQ)

- File: [src/jobs/indexing.job.ts](#)
- Fetches product from Postgres, generates embedding via Ollama (nomic-embed-text), generates BM25 sparse vector



- Upserts to Qdrant with payload: product_id, seller_id, title, price, currency, category_slug, country_code, status, created_at
- Failure: job retried by BullMQ; product remains live regardless

What data is not collected:

- Address/GPS (only region/city dropdowns)
- Seller identity verification at listing time
- Contact number validation

What happens if Qdrant indexing fails:

- Product is live in PostgreSQL, not findable via semantic search
- Only findable via SQL fallback path (keyword contains match)
- No field on Product table to indicate Qdrant sync status
- No alert or retry beyond BullMQ default retries

7. BUYER SEARCH FLOW TRACE

File path map

```
User types query in chat
→ POST /api/chat
(src/app/api/chat/route.ts)
→ getChatSession(sessionId) (src/lib/ai/chat-session.ts → Redis)
→ callOllamaFilters(query)
(src/lib/ai/marketplace-search.ts)
  Redis cache check (key: intent:<query>, TTL 24h)
  → Ollama /api/chat with MARKET_SEARCH_SYSTEM_PROMPT
  → Returns JSON: {intent, keywords, categorySlug, minPrice, maxPrice, condition, sort}
  → parseFiltersFromText(content, query)
(src/lib/ai/marketplace-search.ts)
  Regex fallback category inference
  Price extraction regex
  Rent/sale intent detection
  Smart intent tags (family/luxury/budget/etc)
→ mergeSearchCriteria(session, rawCriteria) (src/lib/ai/chat-session.ts)
  Rolling context: snowballs keywords up to 5
  Category shift detection → reset session
→ searchProductsWithCriteria(criteria, opts)
(src/lib/ai/marketplace-search.ts)
  expandQueryWithOntology(keywords)
(src/lib/search/ontology.ts)
  Redis cache check for embedding (key: embed:<query>, TTL 30d)
  aiProvider.createEmbedding(query) (Ollama nomic-embed-text)
  textToSparseVector(query)
(src/lib/qdrant/client.ts - BM25)
  searchProducts(dense, sparse, filters, limit)
(src/lib/qdrant/client.ts)
```



```
Qdrant → product IDs + scores
prisma.product.findMany({id: {in: productIds}})
getBehaviorScores(productIds) (AnalyticsService →
user_events)
In-memory ranking (8 score components)
Filter by RELEVANCE_CUTOFF = 0.20
↓ If Qdrant fails/empty → SQL fallback
prisma.product.findMany({where: keyword LIKE})
→ MarketplaceEngine.buildSearchResponse()
(src/lib/ai/marketplace-engine.ts)
Deterministic Arabic/English message
Suggestions list
→ SSE stream: products event + delta event + done event
```

Ranking algorithm (8-component scoring)

Component	Weight (general)	Description
A. Semantic relevance	0.40	Qdrant score normalized to max
B. Price value	0.15	Cheaper than median → higher score
C. Freshness	0.10	Days since listing: <1d=1.0, <7d=0.8, <30d=0.5
D. Quality / trust	0.15	Verified seller, images, description length, completenessScore
E. Intent alignment	Additive ±0.30	Intent tag overlap with user tags
F. Price band alignment	Additive ±0.20	cheap/mid/premium match
G. Behavior boost	Additive up to 0.20	Log-scaled click history
H. Admin boost	Additive 0.30	isBoosted flag set by admin

Weights shift dynamically: price-priority → wPrice=0.35, quality-priority → wQual=0.25, freshness-priority → wFresh=0.25.

What can go wrong in buyer search

Failure	Current behavior
Ollama down	callOllamaFilters throws → parseFiltersFromText runs regex-only fallback
Redis down	Cache miss → direct Ollama call; session state lost
Qdrant returns 0	Retries without categorySlug filter once, then falls back to SQL
Qdrant down	Catches error → SQL fallback with LIKE query
Embedding fails	Falls through to SQL
SQL returns 0	deterministicMessage = "No exact matches found. Want me to broaden?"
Ollama timeout	150s timeout (2.5 min) — blocks the user request for that duration



Arabic/English handling

- Arabic detected by Unicode range check: [\u0600-\u06FF]
- Arabic normalized: alef variants → ا, taa marbuta → ة, yaa → ي
- Ontology expansion: Arabic synonyms for common categories (from `src/lib/search/ontology.ts`)
- Qdrant embedding handles both Arabic and English natively (nomic-embed-text is multilingual)
- System prompt instructs Ollama to parse Arabic intent correctly
- Intent cache key: lowercase normalized query (same cache used for Arabic and English)

8. AI / OLLAMA / RUNPOD INTEGRATION ANALYSIS

AI Provider Layer

File	Purpose
src/lib/ai/ai-factory.ts	Singleton factory; reads <code>AI_PROVIDER</code> env; returns <code>OllamaProvider</code> or <code>OpenAIProvider</code>
src/lib/ai/providers/ollama.provider.ts	Implements <code>IAIProvider</code> : <code>createChatCompletionStream</code> , <code>createEmbedding</code> , <code>getDefaultChatModel</code>
src/lib/ai/providers/openai.provider.ts	OpenAI SDK wrapper — fallback only, not used in production
src/lib/ai/providers/ai-provider.interface.ts	Interface contract both providers implement

Every AI call in the system

Function	File	Input	Output	Model	Syn- c?	Blo- cks use r?	Stored in DB?	Goes to Qdrant ?
<code>analyzeImageForListing()</code>	<code>ai-service.ts</code>	Image URL + country code	{title, description, category, condition, priceSuggestion,	llava (vision)	Yes	Yes (block sell flow)	Partial (in chat metadata)	No



Function	File	Input	Output	Model	Sy nc?	Blo cks use r?	Stored in DB?	Goe s to Qdr ant ?
callOllamaFileters()	marketplace-search.ts	User query string	attributes} JSON: {intent , keyword s, categorySlug, price, sort}	qwen2 .5:14 b-instr uct	Yes	Yes (block search)	Via SearchLog	No
aiProvider.createEmbedding()	marketplace-search.ts	Expanded query string	number[] (768-dim)	nomic -embed -text	Yes	Yes (block search)	Cached in Redis 30d	Used to query Qdrant
moderateContent()	ai-service.ts	title + desc + image s + country	{approved, reason, score}	qwen2 .5:14 b-instr uct	No (as ync)	No	In product.attributes.ai_moderation_report	No
generateAiDescription()	ai-service.ts	Product fields	{description, descriptionAr}	qwen2 .5:14 b-instr uct	Backfil scri pt only	No	product.description + descriptionAr	Triggers rein dex
extractVehicleDetails()	vehicle-extractor.ts	title + description	{make, model, year, mileage , transmission, fuelType}	qwen2 .5:14 b-instr uct	Import scri pt only	No	VehicleDetails model	No
generateSystemPrompt()	ai-service.ts	context	System prompt string	None	Yes	No No		No
Chat fallback streaming	api/chat/route.ts	Conversation string	Streamed text	qwen2 .5:14 b-	Yes	Yes	ChatMessage	No



Function	File	Input	Output	Model	Sy nc? cks use r?	Stored in DB?	Goe s to Qdr ant ?
		n histor y		instr uct			

Timeout and retry configuration

Call	Timeout	Retries
callOllamaFilters (intent)	150s	0 (fail fast to fallback)
analyzeImageForListing (vision)	120s	3 (exponential backoff, base 1s)
createEmbedding (embedding)	30s (in reindex script)	4 (in reindex script)
moderateContent	60s	3
Chat fallback streaming	No explicit timeout	0

RunPod usage

- RunPod exposes an Ollama-compatible endpoint via a tunnel
- OLLAMA_BASE_URL is set to the RunPod URL in production
- OLLAMA_API_KEY carries the RunPod Bearer token
- No RunPod-specific client — uses standard Ollama HTTP API
- Ollama on RunPod runs: qwen2.5:14b-instruct, nomic-embed-text, llava

Qwen2.5 tool_call format note

Qwen2.5 natively outputs `<tool_call>` XML-like tags when in tool-calling mode. The v2 branch has a fix to intercept and strip this before streaming to the client. This fix is **not confirmed in main** — if it regresses, clients see raw `<tool_call>` tags in chat output.

9. QDRANT / VECTOR SEARCH ANALYSIS

File: <src/lib/qdrant/client.ts>

Collection configuration

Setting	Value
Collection name	products
Dense vector size	768 dimensions
Distance metric	Cosine
Sparse vector	BM25 (text→token→FNV-1a hash→weight mapping)



Setting	Value
Indexing threshold	vectors_count: 0 (index always)
HNSW _m	16
HNSW _{ef_construct}	100

Payload structure stored in Qdrant (per product point)

```
{
  "product_id": "uuid",
  "seller_id": "uuid",
  "title": "string",
  "title_ar": "string",
  "description": "string (first 500 chars)",
  "price": number,
  "currency": "KWD | SAR",
  "category_slug": "vehicles | electronics | ...",
  "country_code": "KW | SA",
  "status": "active | pending | ...",
  "region_id": number,
  "price_band": "cheap | mid | premium",
  "intent_tags": ["family", "luxury", ...],
  "smart_category": "string",
  "property_type": "apartment | villa | ...",
  "listing_type": "rent | sale",
  "is_furnished": boolean,
  "created_at": "ISO timestamp"
}
```

Payload indexes (for fast filter at Qdrant level)

country_code, category_slug, price, status, region_id, price_band, intent_tags, smart_category, property_type, listing_type, is_furnished

Text embedded to generate dense vector

Combination of: title + title_ar + description + category_slug + optional LLM-generated semantic keywords (reindex script only)

What happens when product changes

- product.status updated → indexing.job.ts enqueued with action: 'update'
- Job fetches new product, re-generates embedding, re-upserts to Qdrant
- **No confirmation** that Qdrant was updated successfully (no embeddingStatus field on Product)

What happens when product is deleted

- indexing.job.ts enqueued with action: 'delete'
- Qdrant point deleted by product.id
- If Qdrant delete fails silently, stale point remains → deleted listing can appear in search results



Risks of stale Qdrant data

- Products deleted from Postgres but still in Qdrant: shown in search, 404 when clicked
- Products updated but job failed: stale title/price shown in chat results
- Products never indexed (job failed): only findable via SQL fallback
- `qdrantPointId` column on Product is **never written** — cannot check "is this indexed?" from DB alone

Scripts to rebuild Qdrant

Script	Action	Safety
<code>scripts/reindex-qdrant.ts</code>	Full rebuild: deletes collection then rebuilds	DESTRUCTIVE — causes zero-result window
<code>scripts/fast-reindex.ts</code>	Partial: upserts only, no delete, 10 concurrent	SAFE
<code>scripts/reindex-missing-qdrant.ts</code>	Indexes products with no Qdrant point	SAFE
<code>scripts/retry-qdrant-failures.ts</code>	Retries failed indexing jobs	SAFE
<code>src/app/api/admin/reindex-all/route.ts</code>	Admin-triggered full reindex (POST)	Exists only in 03042026 branch

10. MULTI-AGENT READINESS TABLE

Seller-Side Agents

Agent	Status	Existing Files	Current Behavior	Required Future	DB Fields Needed	Priority
Publishing Agent	✅ Implemented	<code>api/products/route.ts</code>	Creates product as active immediately	Should publish as pending, hand off to pipeline	—	P0: Fix status race
Category Agent	✅ Partial	<code>ai-service.ts:analyzeImageForListing,</code> <code>marketplace-search.ts:parseFiltersFromText</code>	Vision model guesses category; regex fallback	Dedicated async agent with confidence threshold; human escalation if <60%	<code>categoryAgentConfidence</code> DECIMAL	P1
Image Validation Agent	❌ Missing	None	Images stored without validation	Check: real product image, not logo/screenshot/copyrighted, correct category	<code>imageValidationStatus</code> ENUM	P1



Agent	Status	Existing Files	Current Behavior	Required Future	DB Fields Needed	Priority
Spam Detect Agent	Partial 	Prohibited keywords check (synchronous)	Keyword blocklist, non-contextual	ML-based spam scoring with Arabic support	spamScore DECIMAL, spamReason TEXT	P1
Duplicate Detect Agent	Missing 	None	No dedup — same listing can be posted 100x	Image hash + text similarity check	duplicateScore DECIMAL, duplicateOf UUID	P2
Data Structuring Agent	Partial 	listing-generator.ts:extractAttributes, vehicle-extractor.ts	Extracts storage/color/model from user text	Full structured extraction for all categories	Use existing CategoryAttribute + ProductAttributeValue	P1
AI Description Agent	Implemented 	ai-service.ts:generateAiDescription, scripts/backfill-ai-descriptions.ts	Backfill script only, not triggered on new listings	Auto-run async on every new listing	aiDescription Status ENUM	P2
Embedding Agent	Implemented 	jobs/indexing.job.ts	BullMQ job: embed + upsert to Qdrant	Same, but add status tracking	embeddingStatus ENUM, embeddingError TEXT	P0: Add status field
Qdrant Sync Agent	Partial 	jobs/indexing.job.ts, lib/search/product-index-sync.ts (03042026)	Enqueued on create/update/delete	Same, add health check + recovery	lastSyncedAt TIMESTAMP	P1

Buyer-Side Agents

Agent	Status	Existing Files	Current Behavior	Required Future	DB Fields Needed	Priority
Buyer Intent Agent	Implemented 	marketplace-search.ts:callOllamaFilters	Ollama intent parse with 150s timeout	Same + Arabic dialect handling + caching improvement	—	P0: Reduce timeout



Agent	Status	Existing Files	Current Behavior	Required Future	DB Fields Needed	Priority
Keyword Extract Agent	✓ Implemented	marketplace-search.ts:parseFiltersFromText	Regex + ontology expansion	LLM-extracted semantic keywords stored	SearchLog.normalizedQuery	P1
Search Planning Agent	✓ Partial	marketplace-search.ts:searchProductsWithCriteria	One function handles everything	: plan (filters) → execute (Qdrant) → rank	—	P2
Matching Agent	✓ Implemented	lib/qdrant/client.ts:searchProducts	Hybrid dense+sparse search	Same	—	P1
Ranking Agent	✓ Implemented	marketplace-search.ts (8-component scorer)	In-memory multi-factor ranking	Move to dedicated service, add A/B testing hooks	—	P2
Result Explanation Agent	✓ Partial	marketplace-engine.ts:buildSearchResponse	Deterministic templates	LLM-generate explanation of why results match	—	P3
Query Logging / Learning Agent	✓ Partial	SearchLog, AiSearchRequestLog, UserEvent, SearchTrend	Logs exist, not used for feedback loop	Use click data to retrain ranking weights	SearchTrend.count used	P2

11. BACKGROUND JOBS / SCRIPTS ANALYSIS

Active BullMQ queues (via `scripts/start-worker.ts`)



Queue	Concurrency	Handler	Safe to run
indexing	5	src/jobs/indexing.job.ts	Yes — idempotent upsert
notification	5	src/jobs/notification.job.ts	Yes
file	2	src/jobs/file.job.ts	Yes
system	1	src/jobs/system.job.ts	Yes (with caution on cleanup)

Operational scripts

Script	Purpose	Command	Idempotent	Data Risk	Prod Safe
scripts/reindex-qdrant.ts	Full Qdrant rebuild with LLM keywords	tsx --env-file=.env scripts/reindex-qdrant.ts	Partial (--resume flag)	HIGH (deletes collection)	⚠ Off-hours only
scripts/fast-reindex.ts	Quick upsert without LLM	tsx --env-file=.env scripts/fast-reindex.ts	Yes	Low	✅
scripts/reindex-missing-qdrant.ts	Index products not yet in Qdrant	same pattern	Yes	Low	✅
scripts/backfill-ai-descriptions.ts	Generate AI descriptions for short listings	tsx ... --dry-run first	Partial	Medium (overwrites descriptions)	✅ with --dry-run first
scripts/backfill-smart-fields.ts	Fill smartCategory/priceBand/intentTags	tsx ...	Yes (only fills nulls)	Low	✅
scripts/import-q84-data.ts	Import Q84sale scraped data	tsx ...	Yes (skips duplicates)	Medium (creates fake users)	⚠ Test first
scripts/seed-beta-data.ts	Seed 10 test listings	tsx ...	Yes	Low	✅ (dev only)
scripts/fast-reindex.ts	Fast upsert	same	Yes	Low	✅
scripts/backfill-ai-qdrant-raw.ts	Backfill Qdrant without LLM enrichment	same	Yes	Low	✅
scripts/patch-qdrant-smart-fields.ts	Update existing Qdrant payloads with enrichment	same	Yes	Low	✅



Script	Purpose	Command	Idempotent	Data Risk	Prod Safe
scripts/create-admin.ts	Create admin user	same	Yes (upsert)	Low	✓
scripts/evaluate-intents.ts	Test intent parsing quality	same	Yes (read-only)	None	✓

12. ADMIN DASHBOARD ANALYSIS

What admin can currently see

- **Dashboard metrics:** total users, total listings, pending listings, inquiries, reports, featured listings
- **Product catalog:** all products paginated, filterable by status/seller/category/country/text search
- **Pending listings:** products awaiting moderation
- **Seller management:** list/approve/reject/suspend sellers
- **User management:** list, view, update, delete users
- **Category management:** CRUD for categories and their dynamic attributes
- **Location management:** CRUD for country/region/city/district hierarchy
- **Package management:** subscription packages CRUD
- **Promotions:** promote listing to featured/homepage/category_boost
- **Reports:** view abuse reports, resolve/dismiss
- **Audit logs:** who did what and when
- **Login activity:** IP/device/location history per user
- **Search logs:** what users searched
- **Contact logs:** call/WhatsApp clicks per listing
- **Analytics:** per-module analytics endpoints (/api/admin/analytics/[module])

What admin CANNOT see (missing)

- **Qdrant sync status** per listing — no way to know if a product is indexed
- **AI moderation results** per listing — ai_moderation_report buried in attributes JSON
- **Embedding generation status** — no field to show
- **Agent pipeline status** — no concept yet
- **Spam score** per listing — no field
- **Failed AI jobs** — no visibility into BullMQ failures beyond Bull Board UI (port 4501)
- **Bull Board UI** only accessible on worker port 4501 — not integrated into admin panel

13. SECURITY FINDINGS

 CRITICAL**C1: Products go live before AI moderation completes**

- File: [src/app/api/products/route.ts](#) lines ~432–551
- Product created with `status: 'active'` inside `$transaction()`. Async moderation runs AFTER. If moderation fails/times out, prohibited item is live indefinitely.
- Fix: Publish as `pending`, move to `active` only after moderation passes.

C2: Image proxy has SSRF vulnerability

- File: [src/app/api/images/proxy/route.ts](#) lines ~103–117
- Accepts any `http://` or `https://` URL. Attacker can probe `http://169.254.169.254/`, internal Postgres/Redis/Qdrant ports.
- Fix: Whitelist allowed CDN domains only.

C3: Auto-verify unverified users at login

- File: [src/app/api/auth/login/route.ts](#) lines ~84–94
- Anyone who registers any email gets auto-verified on first login. No email ownership check.
- Fix: Remove MVP bypass, implement real email verification flow.

C4: Unrestricted image URL passed to LLaVA

- File: [src/app/api/chat/route.ts](#) line ~270
- `image_url` from request body passed directly to `analyzeImageForListing()`. Can be any URL including internal services.
- Fix: Validate URL is from known S3 bucket or CDN before sending to LLaVA.

 HIGH**H1: Wildcard CORS on all API routes**

- File: [src/middleware.ts](#), [next.config.ts](#)
- `'Access-Control-Allow-Origin': '*'` set on API routes. Cross-origin requests from any site are allowed.
- Fix: Set specific origin from `NEXT_PUBLIC_SITE_URL` env var.

H2: DEV_AUTH_BYPASS in non-production environments

- Files: `middleware.ts`, `rate-limiter.ts`, `rate-limiter-node.ts`
- If `DEV_AUTH_BYPASS=true` is deployed to staging/prod, all rate limiting is disabled.
- Fix: Assert this env var is always `false` at startup in non-dev environments.

H3: Seller auto-creation — any user becomes a seller

- File: [src/app/api/products/route.ts](#) lines ~267–295



- Any authenticated user calling `POST /api/products` gets a Seller record + seller role assigned automatically. No approval, no KYC, no gate.
- Fix: Add explicit seller registration with admin approval flow.

H4: Auth verbose logging — emails in plaintext logs

- File: [src/app/api/auth/login/route.ts](#)
- `console.log('LOGIN look up user', email)` logs email address of every login attempt in plaintext.
- Fix: Remove all `console.log` from auth routes; use structured Pino logger at DEBUG level.

H5: Weak token generation using `Math.random()`

- File: [src/lib/auth.ts](#) lines ~56–63
- Verification/reset tokens generated with `Math.random()` — not cryptographically secure.
- Fix: Use `crypto.randomBytes(32).toString('hex')`.

H6: Internal `fetch()` from chat to products API

- File: [src/app/api/chat/route.ts](#) lines ~565–587
- Chat publishes listings by making an HTTP `fetch()` call to its own `/api/products` endpoint, forwarding cookies/headers. No timeout; no fallback if the internal call hangs.
- Fix: Call Prisma directly or use a service layer function; add `AbortSignal` timeout.

H7: S3 key extracted via naive string split

- File: [src/app/api/products/route.ts](#) line ~462
- `url.split('/').slice(-2).join('/')` — malleable with crafted URLs.
- Fix: Parse URL properly with `new URL(url)` and extract pathname.



MEDIUM

M1: Prohibited keywords fetched from DB on every product create — no cache

- Fix: Cache in Redis with 5-minute TTL.

M2: Async moderation silently fails — no retry

- Fix: Use BullMQ job with retries instead of fire-and-forget.

M3: Image URLs not validated against own S3 bucket before storing

- Fix: Check `new URL(url).hostname.endsWith(AWS_S3_BUCKET + '.s3.amazonaws.com')`.

M4: `docker.internal` hostname replaced via string substitution in chat route



- File: [src/app/api/chat/route.ts](#) lines ~456, 639
- `String(url).replace('host.docker.internal', 'localhost')` — fragile, could expose internal URLs in other environments.

M5: Rate limiting on auth endpoints too lenient

- 5 attempts per 15 minutes = 20/hour = 480/day per IP. Brute force feasible for short passwords.

M6: No CSRF protection on state-changing routes

- `SameSite=Lax` on cookies mitigates some vectors but doesn't fully protect POST/DELETE.

14. PERFORMANCE / SCALABILITY FINDINGS

Current bottlenecks

Bottleneck	Impact	File
Ollama intent parse: 150s timeout in request cycle	Search hangs 2.5 min if Ollama is slow	marketplace-search.ts:callOllamaFilters
Embedding in request cycle	Search hangs on cold embed cache	marketplace-search.ts:searchProductsWithCriteria
Prohibited keywords DB query per product create	Slow listing creation at scale	api/products/route.ts
No Postgres indexes on title <code>LIKE %keyword%</code>	Full table scan on SQL fallback	marketplace-search.ts SQL fallback
<code>prisma.product.findMany</code> in search: fetches all fields	Over-fetch; could use smaller select	marketplace-search.ts line ~372
<code>getBehaviorScores</code> query per search	Extra DB round-trip on every search	analytics.service.ts



Bottleneck	Impact	File
Qdrant reindex script: sequential embedding (1 at a time)	1000 products = hours	scripts/reindex-qdrant.ts

What breaks at 10k listings

- SQL fallback with `title LIKE '%keyword%'` becomes unusable (full table scan, no GIN/GiST index)
- `prohibited keywords` fetch from DB every create — slow, add Redis cache
- `getBehaviorScores` query hits `user_events` table without proper time-bounded index
- Qdrant `products` collection with 10k points: search is still fast (HNSW), no issue

What breaks at 100k listings

- Qdrant at 100k with 768-dim vectors: ~300MB memory — fine
- Postgres with 100k products: `findMany` for all in a category gets slow without category index (already indexed)
- `reindex-qdrant.ts` sequential script: at 1 product/sec = 27 hours for 100k
- Chat session Redis: 10 sessions/user × 100k users = 1M keys, each ~500 bytes = 500MB — Redis handles it

What should move to background jobs

- AI description generation
- Smart enrichment calculation
- Qdrant sync (already in queue, but no status feedback)
- AI moderation (currently fire-and-forget, needs proper queue)
- Image re-sizing / thumbnail generation (currently in `file.job.ts` but not triggered)

Caching opportunities

Cache	Current	Suggested
Ollama intent responses	Redis 24h TTL	Already done
Embeddings	Redis 30-day TTL	Already done
Prohibited keywords	Not cached	Redis 5-min TTL
Category tree	Not cached	Redis 1-hour TTL
Product list pages	Not cached	Redis 30s TTL for public pages
Seller subscription status	Not cached	Redis 5-min TTL

15. LOGGING / MONITORING FINDINGS

What is logged now



- Pino structured logger configured in `src/lib/logger.ts` with redaction for: password, token, refreshToken, authorization
- Performance logs: `[Performance]` Ollama intent parsing took Xms, Qdrant returned N items, Marketplace search (Qdrant path) took Xms
- BullMQ job lifecycle: completed, failed, error events in `src/lib/queue/workers.ts`
- AI moderation errors: `console.error('[Async AI] Moderation Wrapper Failed', ...)`
- Qdrant errors: `console.error('Qdrant semantic search failed. Falling back to SQL:', ...)`
- Auth verbose: `console.log('LOGIN look up user', email)` — **security risk**

What is missing

- Structured search logs written to `SearchLog` table — only done in `AiSearchRequestLog`, not on every search
- Failed Qdrant indexing not logged to DB — only BullMQ job failure log
- Agent pipeline result logs — no `AiToolCall` records for most operations (model exists, not populated)
- No APM / distributed tracing (no OpenTelemetry, no Datadog, no Sentry)
- No health dashboard for Ollama/RunPod availability
- No alerting when Ollama is down (fallback happens silently)
- Sensitive data in logs: email addresses, raw query text containing PII

AiToolCall model is defined but barely populated

The `AiToolCall` table in Prisma tracks per-invocation latency, tokens, and result. This is the foundation for multi-agent observability — but only the `03042026` branch (not `main`) populates it.

16. TESTING FINDINGS

Existing tests

File	Type	What it covers
<code>tests/ai/marketplace-search.test.ts</code>	Unit	Intent parsing, filter extraction
<code>tests/e2e/ai-search.spec.ts</code>	Playwright E2E	Search flow from browser
<code>tests/e2e/buyer-journey.spec.ts</code>	Playwright E2E	Full buyer flow
<code>tests/e2e/seller-journey.spec.ts</code>	Playwright E2E	Full seller flow
<code>tests/e2e/verify-api.spec.ts</code>	Playwright E2E	API endpoint verification
<code>tests/e2e/verify-flow.spec.ts</code>	Playwright E2E	Flow verification
<code>tests/api-test.js</code>	Manual script	Raw API calls
<code>tests/chat-test.js</code>	Manual script	Chat endpoint
<code>tests/check-qdrant.js</code>	Manual script	Qdrant health check

Note: The `03042026` branch has 12 additional unit/integration tests that are NOT in `main`:



- tests/api/auth-refresh.test.ts
- tests/api/chat-route.test.ts
- tests/api/chat-streaming-phases.test.ts
- tests/api/products-search-route.test.ts
- tests/security/vision-image-url.test.ts
- tests/jobs/indexing.job.test.ts
- tests/search/product-index-sync.test.ts
- and more

Commands to run

```
npm run typecheck          # tsc --noEmit
npm run lint                # next lint
npx playwright test        # E2E (requires running server)
npx vitest                  # Unit tests (vitest configured)
```

Biggest gaps without tests

- Sell flow end-to-end (chat → LLaVA → draft → publish) — currently only Playwright, no unit test
- Qdrant sync on product create/update/delete
- AI moderation race condition
- Arabic intent parsing correctness
- Ranking algorithm (no unit test for 8-component scorer)

17. MOBILE APP FINDINGS

Structure

```
ios/PickpickMobile/
  project.yml                      XcodeGen config
  PickpickMobile/
    Models/                        APIModels.swift, Auth.swift,
Product.swift, Category.swift
  Services/                        APIClient.swift, AuthService.swift,
ProductService.swift, etc.
  ViewModels/                     AIChatViewModel.swift,
AuthViewModel.swift, HomeViewModel.swift
  Views/                          AIChatView.swift, HomeView.swift,
ProductDetailView.swift, InboxView.swift
  Utils/                          AppConfig.swift, AppTheme.swift,
PriceFormatter.swift
```

Critical mismatches with web backend

Issue	Impact
Mobile uses Bearer JWT; web uses cookie JWT	Both work against same backend, but different auth mechanisms — could cause session confusion



Issue	Impact
Mobile sends no location: {country_code, language} in chat	Backend defaults to <code>KW</code> and <code>en</code> — Arabic locale never set from mobile
Mobile only handles 5 SSE events; web handles 8	Mobile ignores <code>status</code> , <code>analysis</code> , <code>intent_picker</code> , <code>seller_title_prompt</code> events
Mobile listing draft: <code>image_urls: [String];</code> web: <code>images: [{previewUrl, s3Url}]</code>	Different deserialization — backend must send mobile-compatible format
Mobile has no chat persistence call	Chat history not saved to DB from mobile sessions
Mobile has no bilingual fields	<code>titleAr</code> , <code>descriptionAr</code> never sent from mobile listing submissions
Mobile <code>NotificationsView.swift</code> exists but not wired (untracked file)	Push notifications not functional
Mobile <code>LocationFetcher.swift</code> exists (untracked)	GPS location not sent to search

Build status

- `ios/PickpickMobile/build/` contains compiled artifact — sim build succeeded at some point
- `ExportOptions.plist` and `ExportOptionsTestFlight.plist` exist — TestFlight distribution was attempted
- The app is production-code-quality SwiftUI, not a prototype

18. MAJOR RISKS

Risk	Severity	Description
Moderation race condition	Critical	Items published publicly before AI moderation — prohibited content can go live
SSRF in image proxy	Critical	Attacker can probe internal network via <code>/api/images/proxy</code>
No embedding status tracking	High	Can't know which products are searchable via Qdrant
Stale Qdrant data	High	Deleted products can appear in search results
Full reindex is destructive	High	Running <code>reindex-qdrant.ts</code> causes zero-result window
Seller auto-creation	High	Any user can publish listings without approval
Auto-verify at login	High	Email verification bypass never removed post-MVP
Mobile/web auth divergence	Medium	Two auth mechanisms against same backend — maintenance complexity
150s Ollama timeout blocks request	High	Search can hang for 2.5 min if Ollama is slow



Risk	Severity	Description
03042026 branch not merged	Medium	Refresh tokens, SSRF protection, test suite, streaming phases — valuable work not in main
PropertyNir sub-module	Low	KYC + tokenized property investment code in same codebase — regulatory risk for MVP

19. MISSING PIECES

For production stability

- [] `embeddingStatus` field on Product (pending/indexed/failed)
- [] `moderationStatus` field on Product (separate from `status`)
- [] Admin visibility of Qdrant sync state
- [] Async moderation as a proper BullMQ job with retries
- [] Refresh token flow (exists in 03042026, not in main)
- [] SSRF protection on image proxy and vision URL (exists in 03042026)
- [] Remove `DEV_AUTH_BYPASS` from all non-local environments

For multi-agent architecture

- [] Agent orchestrator (decides which agents run on which products)
- [] `AgentLog` table (per-agent invocation: status, latency, output, error)
- [] Per-agent BullMQ job types (currently only `indexing` handles everything)
- [] Duplicate detection logic (image hash + text embedding similarity)
- [] Spam scoring field + agent
- [] Image validation agent (real photo, correct category, not copyrighted)
- [] Category confidence field + agent that can escalate to human review
- [] AI description agent triggered on every new listing (not just backfill script)
- [] Query logging learning loop (clicks → ranking weight adjustment)

For mobile parity

- [] Consistent auth mechanism (standardize on Bearer token or cookie)
- [] Mobile sends `country_code` and `language` in chat requests
- [] Push notification wiring
- [] GPS location sent to search

20. RECOMMENDED TARGET ARCHITECTURE (MULTI-AGENT)

SELLER PATH:

User posts listing

→ Publishing Agent (validates, creates as 'pending')

↓ (async, via BullMQ)



```
→ Image Validation Agent    (real photo? correct cat? → sets  
imageValidationStatus)  
→ Category Agent            (detect category with confidence →  
categoryAgentConfidence)  
→ Spam Detection Agent      (score 0-1 → spamScore)  
→ Duplicate Detection Agent (image hash + text sim → duplicateScore,  
duplicateOf)  
→ Data Structuring Agent    (extract structured attributes →  
ProductAttributeValue)  
→ AI Description Agent       (bilingual desc if short → descriptionAr,  
aiDescriptionStatus)  
→ Embedding Agent           (embed → Qdrant upsert → embeddingStatus =  
'indexed')  
→ Final Gate: all agents passed? → set status = 'active'  
any agent failed? → set status = 'pending' → admin review  
queue
```

BUYER PATH:

User types query

```
→ Buyer Intent Agent        (Ollama parse, cached, 30s timeout max)  
→ Keyword Extraction Agent   (ontology expansion + semantic keywords)  
→ Search Planning Agent     (decide: Qdrant filters, SQL filters, fallback  
strategy)  
→ Matching Agent            (Qdrant hybrid search)  
→ Ranking Agent              (8-component scorer → sorted list)  
→ Result Explanation Agent   (deterministic or LLM-generated explanation)  
→ Query Logging Agent        (log to SearchLog + AiSearchRequestLog +  
SearchTrend)
```

21. STEP-BY-STEP IMPLEMENTATION ROADMAP

Phase 0 — Fix Critical Issues (1–2 weeks)

1. Set status: 'pending' on product create; only activate after async moderation passes
2. Fix SSRF in image proxy (whitelist CDN domains)
3. Remove `DEV_AUTH_BYPASS` from any non-local env
4. Merge `src/lib/security/vision-image-url.ts` from 03042026 branch
5. Remove `console.log('LOGIN look up user', email)` from auth route
6. Fix Qwen tool_call format stripping (from v2 branch)

Phase 1 — Observability & Stability (2–3 weeks)

7. Add `embeddingStatus` ENUM and `moderationStatus` ENUM fields to Product
8. Populate `AiToolCall` table for all AI invocations (from 03042026)
9. Add Qdrant sync status to admin product detail view
10. Move async moderation from fire-and-forget to BullMQ job with retries
11. Add Redis cache for prohibited keywords
12. Merge refresh token flow from 03042026 branch
13. Add `embeddingError` TEXT field, store Qdrant failure reason

Phase 2 — Agent Infrastructure (3–4 weeks)



14. Create `AgentLog` table (`agentType`, `productId`, `status`, `latencyMs`, `resultJson`, `error`)
15. Create dedicated BullMQ job types per agent: `moderation`, `image-validation`, `description`, `embedding`, `category`, `spam`, `duplicate`
16. Create agent orchestrator: triggers correct agents based on product state
17. Create agent pipeline status endpoint for admin
18. Reduce Ollama intent parse timeout from 150s to 30s with faster model fallback

Phase 3 — Agent Implementations (4–6 weeks)

19. Image Validation Agent: verify real product photo, not NSFW/copyrighted
20. Duplicate Detection Agent: compare embedding cosine similarity to recent products
21. Spam Detection Agent: LLM-based spam classifier with Gulf-dialect Arabic support
22. Category Agent with confidence threshold and human escalation
23. AI Description Agent: auto-trigger on every new listing if description <80 chars
24. Data Structuring Agent: full attribute extraction for all category types

Phase 4 — Buyer Intelligence (3–4 weeks)

25. Query Learning Loop: use click data to adjust ranking weights
26. Personalization: per-user preference model from search history
27. Mobile parity: standardize auth, add location sending, fix SSE event handling

22. PRIORITY LIST

Must fix now (production blockers)

- ☐ C1: Moderation race condition — products go live before moderation
- ☐ C2: SSRF in image proxy
- ☐ H2: Remove `DEV_AUTH_BYPASS` from any deployed environment
- ☐ H4: Remove email from login logs
- ☐ H3: Require seller approval — don't auto-create sellers
- ☐ Add `embeddingStatus` to `Product` (can't operate multi-agent without it)

Should fix soon (within 2 weeks)

- ☐ Merge refresh token auth from 03042026
- ☐ Fix CORS to specific origin
- ☐ Move moderation to BullMQ job
- ☐ Cache prohibited keywords
- ☐ Fix Ollama timeout (150s → 30s max with fallback)
- ☐ Merge SSRF URL validator from 03042026
- ☐ Port 03042026 test suite into `main`
- ☐ Add Bull Board UI link to admin panel

Later (Phase 2+)

- ☐ Agent infrastructure (`AgentLog` table, per-agent jobs)



- [] Mobile auth standardization
- [] Duplicate detection
- [] Spam scoring
- [] Full PropertyNir security review (KYC, token transactions)
- [] Add GIN index on `product.title` for SQL fallback
- [] Next.js 16 upgrade (CVE-2025-66478 fix from marketplace-delivery-clean)

23. FILE PATH INDEX

Feature	Files
AI chat entry point	<code>src/app/api/chat/route.ts</code>
Intent parsing (Ollama)	<code>src/lib/ai/marketplace-search.ts:callOllamaFilters</code>
Filter extraction (regex)	<code>src/lib/ai/marketplace-search.ts:parseFiltersFromText</code>
Semantic search + ranking	<code>src/lib/ai/marketplace-search.ts:searchProductsWithCriteria</code>
Chat session state	<code>src/lib/ai/chat-session.ts</code>
Ambiguity detection	<code>src/lib/ai/ambiguity-flow.ts</code>
Response builder (Arabic)	<code>src/lib/ai/marketplace-engine.ts</code>
Image analysis (LLaVA)	<code>src/lib/ai/ai-service.ts:analyzeImageForListing</code>
AI moderation	<code>src/lib/ai/ai-service.ts:moderateContent</code>
Listing generator (user facts)	<code>src/lib/ai/listing-generator.ts</code>
Vehicle extractor	<code>src/lib/ai/vehicle-extractor.ts</code>
Arabic normalization	<code>src/lib/ai/normalize-search-query.ts</code>
Ontology expansion	<code>src/lib/search/ontology.ts</code>
Qdrant client (search/upsert)	<code>src/lib/qdrant/client.ts</code>
Smart enrichment	<code>src/lib/marketplace/enrichment.ts</code>
Visibility filter	<code>src/lib/marketplace/visibility.ts</code>
Product creation API	<code>src/app/api/products/route.ts</code>



Feature	Files
Listing creation API	src/app/api/listings/route.ts
Seller form UI	src/app/(seller)/sell/new/page.client.tsx
Listing form component	src/components/seller/listing-form.tsx
Upload presign	src/app/api/upload/presign/route.ts
JWT auth	src/lib/auth/jwt.ts
Rate limiting	src/lib/rate-limiter.ts, src/lib/rate-limiter-node.ts
RBAC	src/lib/rbac.ts, src/lib/auth/roles.ts
BullMQ jobs	src/jobs/indexing.job.ts, notification.job.ts, file.job.ts, system.job.ts
Worker startup	scripts/start-worker.ts
Admin dashboard API	src/app/api/admin/dashboard/route.ts
Admin moderation	src/app/api/admin/moderation/listings/[id]/approve/route.ts
Analytics	src/lib/services/analytics.service.ts
Audit logging	src/lib/services/audit-service.ts
iOS app	ios/PickpickMobile/PickpickMobile/
iOS chat streaming	ios/.../ViewModels/AIChatViewModel.swift
iOS API client	ios/.../Services/APIClient.swift
Reindex Qdrant	scripts/reindex-qdrant.ts
Backfill descriptions	scripts/backfill-ai-descriptions.ts
Import Q84 data	scripts/import-q84-data.ts
Docker (dev)	docker-compose.yml
Docker (prod)	docker-compose.prod.yml
Prisma schema	prisma/schema.prisma

24. OPEN QUESTIONS

1. **Seller approval flow:** Should sellers be approved by admin before listing? Currently auto-approved — is this intentional for MVP?
2. **Email verification:** Is the auto-verify-at-login bypass intentional for MVP, and when will it be removed?
3. **PropertyNir:** Is real estate tokenization in scope for MVP or should it be isolated/disabled until legal review?
4. **Qdrant collection:** Is the current 768-dim `nomic-embed-text` embedding the permanent choice, or will it be swapped? (Swapping means full reindex.)



5. **RunPod permanence:** Is RunPod the long-term AI hosting solution, or transitional? A dedicated GPU server would remove the tunnel latency.
 6. **Mobile auth:** Should mobile use cookie auth (align with web) or should the backend support both? Currently both work but with different session behaviors.
 7. **Multi-language:** Is English a first-class language or secondary? The system supports both but Arabic is prioritized — how should the agent pipeline handle English-only listings?
 8. **03042026 branch:** Are the features there (refresh tokens, SSRF fix, test suite) approved for merge? They are significantly more mature than what's in `main`.
 9. **Moderation threshold:** What is the acceptable time window between a listing going live and being moderated? 0 seconds (block publish until moderated) or minutes/hours?
 10. **Agent pipeline ordering:** Should all agents run in parallel (faster) or sequentially (simpler, easier to debug)?
-

CHATGPT PASTE SUMMARY

Copy this section when briefing ChatGPT:

This is a Next.js 15 AI-powered classifieds marketplace (SellSell/PickPick) for Kuwait/Gulf markets, Arabic-first, RTL UI. The backend is Node.js with Prisma/PostgreSQL. AI runs on Ollama (local or RunPod) using qwen2.5:14b-instruct for intent/moderation, llama for image analysis, nomic-embed-text for embeddings. Vector search uses Qdrant with hybrid dense+sparse search. Session state is in Redis. Background jobs use BullMQ in a separate worker container. There is also a native iOS SwiftUI app.

The chat system is a Redis-backed state machine, not LLM-for-every-message. Intent parsing calls Ollama, results are cached in Redis 24h. Embeddings cached 30 days. The ranking is an 8-component in-memory scorer (semantic + price + freshness + quality + intent alignment + price band + behavior + admin boost).

Critical problems today:

1. Products go live before AI moderation completes (race condition — status = 'active' before moderation async job finishes)
2. SSRF vulnerability in image proxy
3. Users auto-verified at login (MVP bypass never removed)
4. No field to track Qdrant sync status per product
5. Stale Qdrant data if delete job fails
6. Mobile app uses Bearer JWT; web uses cookie JWT (same backend)
7. 150-second Ollama timeout blocks user search requests

For multi-agent architecture: The foundation is all there (BullMQ, Prisma, Qdrant, Ollama) but there is no agent orchestrator, no per-agent job types, no `AgentLog` table, and no `embeddingStatus` / `moderationStatus` fields on the Product model. These are the first things to add before implementing individual agents.



Branches: Main branch is production. The 03042026 branch has refresh tokens, SSRF protection, a 12-test test suite, and streaming phase events — these should be merged. The kuwait branch is the original Firebase/Google Maps prototype (~100 commits, never merged). The marketplace-delivery-clean branch has the Next.js 16 CVE fix.